Experiment 01 | Breadth First Search (BFS)

Uninformed Search · Graph Traversal · docs/bfs.md · src/bfs.py

Aim

To implement Breadth First Search in Python and use it to find the shortest path (in number of edges) between a start node and a goal node in an unweighted graph.

Theory

Breadth First Search is an uninformed graph traversal algorithm that explores a graph level by level. It starts at a source node and visits all of its immediate neighbors before moving on to the neighbors of those neighbors. This level-order expansion is achieved using a queue (FIFO), which guarantees that BFS finds the shortest path in an unweighted graph.

Algorithm

- Add the start node to a queue and mark it as visited.
- While the queue is not empty, dequeue the front node.
- If the dequeued node is the goal, stop and return the path.
- Otherwise, enqueue all unvisited neighbors and mark them visited.
- Repeat until the goal is found or the queue becomes empty.

Python Implementation

```
from collections import deque

def bfs(graph, start, goal):
    visited = {start}
    queue = deque([[start]])

    while queue:
        path = queue.popleft()
        node = path[-1]

        if node == goal:
            return path

        for neighbor in graph.get(node, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(path + [neighbor])

    return None
```

Sample Input

```
graph = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A", "E"],
    "D": ["B", "E"],
    "E": ["C", "D"],
}
start_node = "A"
goal_node  = "E"
```

Output

```
Running BFS from 'A' to 'E'...

Path found:
A -> C -> E
```

Complexity Analysis

Metric	Complexity	Notes
Time	O(V + E)	Every vertex and edge is visited at most once
Space	O(V)	Queue and visited set can hold up to all vertices

Applications

- Shortest path in unweighted graphs
- Web crawling and link traversal
- Social network degree-of-connection queries
- Broadcasting in peer-to-peer networks